

Relative Files.

Creating a Relative File

```
      $ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID.    CreateRelativeFromSeq.
* Creates a Relative file from a sequential file.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.

    SELECT SupplierFile ASSIGN TO "SUPP.DAT"
        ORGANIZATION IS RELATIVE
        ACCESS MODE IS RANDOM
        RELATIVE KEY IS SupplierKey
        FILE STATUS IS SupplierStatus.

    SELECT SupplierFileSeq ASSIGN TO "INSUPP.DAT".

DATA DIVISION.
FILE SECTION.
FD SupplierFile.
01 SupplierRecord.
    02 SupplierCode          PIC 99.
    02 SupplierName          PIC X(20) .
    02 SupplierAddress       PIC X(60) .

FD SupplierFileSeq.
01 SupplierRecordSeq.
    88 EndOfFile          VALUE HIGH-VALUES.
    02 SupplierCodeSeq     PIC 99.
    02 SupplierNameSeq     PIC X(20) .
    02 SupplierAddressSeq  PIC X(60) .

WORKING-STORAGE SECTION.
01 SupplierStatus          PIC X(2) .
01 SupplierKey             PIC 99.
```

Creating a Relative File

```
$ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID. CreateRelativeFromSeq.
* Creates a Relative file from a sequential file.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
    SELECT SupplierFile ASSIGN TO "SUPP.DAT"
        ORGANIZATION IS RELATIVE
        ACCESS MODE IS RANDOM
        RELATIVE KEY IS SupplierKey
        FILE STATUS IS SupplierStatus.
    SELECT SupplierFileSeq ASSIGN TO "INSUPP.DAT".

DATA DIVISION.
FILE SECTION.
FD SupplierFile.
01 SupplierRecord.
    02 SupplierCode          PIC 99.
    02 SupplierName          PIC X(20).
    02 SupplierAddress       PIC X(60).

FD SupplierFileSeq.
01 SupplierRecordSeq.
    88 EndOfFile          VALUE HIGH-VALUES.
    02 SupplierCodeSeq     PIC 99.
    02 SupplierNameSeq     PIC X(20).
    02 SupplierAddressSeq  PIC X(60).

WORKING-STORAGE SECTION.
01 SupplierStatus          PIC X(2).
01 SupplierKey              PIC 99.
```

The diagram illustrates the relationship between variables defined in the FILE-CONTROL and WORKING-STORAGE sections. A green arrow originates from the `SupplierKey` variable in the `FILE-CONTROL` section and points to the `SupplierKey` variable in the `WORKING-STORAGE SECTION`. A red arrow originates from the `SupplierStatus` variable in the `FILE-CONTROL` section and points to the `SupplierStatus` variable in the `WORKING-STORAGE SECTION`.

Creating a Relative File

```
PROCEDURE DIVISION.  
Begin.  
  OPEN OUTPUT SupplierFile.  
  OPEN INPUT SupplierFileSeq.  
  
  READ SupplierFileSeq  
    AT END SET EndOfFile TO TRUE  
  END-READ  
  PERFORM UNTIL EndOfFile  
    MOVE SupplierCodeSeq TO SupplierKey  
    MOVE SupplierRecordSeq TO SupplierRecord  
    WRITE SupplierRecord  
    INVALID KEY DISPLAY "SUPP STATUS :-" , SupplierStatus  
  END-WRITE  
  READ SupplierFileSeq  
    AT END SET EndOfFile TO TRUE  
  END-READ  
  END-PERFORM.  
  
CLOSE  SupplierFile, SupplierFileSeq.  
STOP RUN.
```

Reading a Relative File.

```
$ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID.    ReadRelative.
* Reads a Relative file directly or in sequence

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
SELECT SupplierFile ASSIGN TO "SUPP.DAT"
    ORGANIZATION IS RELATIVE
    ACCESS MODE IS DYNAMIC
    RELATIVE KEY IS SupplierKey
    FILE STATUS IS SupplierStatus.

DATA DIVISION.
FILE SECTION.
FD SupplierFile.
01 SupplierRecord.
   88 EndOfFile    VALUE HIGH-VALUES.
   02 SupplierCode          PIC 99.
   02 SupplierName         PIC X(20).
   02 SupplierAddress      PIC X(60).

WORKING-STORAGE SECTION.
01 SupplierStatus          PIC X(2).
   88 RecordFound        VALUE "00".
01 SupplierKey             PIC 99.
01 PrnSupplierRecord.
   02 PrnSupplierCode      PIC BB99.
   02 PrnSupplierName      PIC BBX(20).
   02 PrnSupplierAddress   PIC BBX(50).
01 ReadType                PIC 9.
   88 DirectRead          VALUE 1.
   88 SequentialRead      VALUE 2.
```

Reading a Relative File.

```
PROCEDURE DIVISION.  
BEGIN.  
  OPEN INPUT SupplierFile.  
  DISPLAY "Enter Read type (Direct=1, Seq=2)-> " WITH NO ADVANCING.  
  ACCEPT ReadType.  
  
  IF DirectRead  
    DISPLAY "Enter supplier key (2 digits)-> " WITH NO ADVANCING  
    ACCEPT SupplierKey  
    READ SupplierFile  
      INVALID KEY DISPLAY "SUPP STATUS :- ", SupplierStatus  
    END-READ  
    PERFORM DisplayRecord  
  END-IF  
  
  IF SequentialRead  
    READ SupplierFile NEXT RECORD  
      AT END SET EndOfFile TO TRUE  
    END-READ  
    PERFORM UNTIL EndOfFile  
      PERFORM DisplayRecord  
      READ SupplierFile NEXT RECORD  
        AT END SET EndOfFile TO TRUE  
      END-READ  
    END-PERFORM  
  END-IF  
  
  CLOSE SupplierFile.  
  STOP RUN.
```

```
DisplayRecord.  
  IF RecordFound  
    MOVE SupplierCode TO PrnSupplierCode  
    MOVE SupplierName TO PrnSupplierName  
    MOVE SupplierAddress TO PrnSupplierAddress  
    DISPLAY PrnSupplierRecord  
  END-IF.
```

Reading a Relative File.

RUN OF REL-EG2.EXE USING SEQUENTIAL READING

Enter Read type (Direct=1, Seq=2)-> 2

01	VESTRON VIDEOS	OVER THE SEA SOMEWHERE IN LONDON
02	EMI STUDIOS	HOLLYWOOD, CALIFORNIA, USA
03	BBC WILDLIFE	BUSH HOUSE, LONDON, ENGLAND
04	CBS STUDIOS	HOLLYWOOD, CALIFORNIA, USA
05	YACHTING MONTHLY	TREE HOUSE, LONDON, ENGLAND
06	VIRGIN VIDEOS	IS THIS ONE ALSO LOCATED IN ENGLAND
07	CIC VIDEOS	NEW YORK PLAZZA, NEW YORK, USA

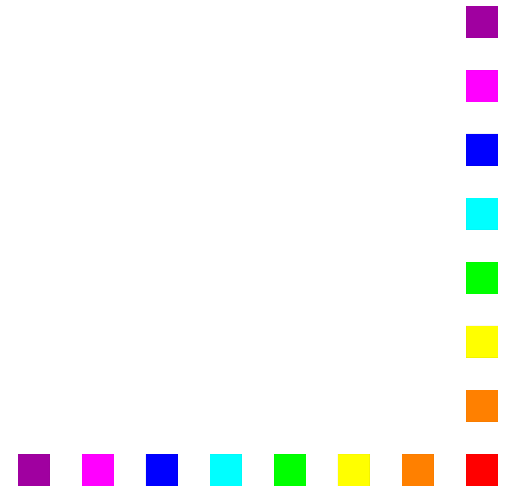
RUN OF REL-EG2.EXE USING DIRECT READ

Enter Read type (Direct=1, Seq=2)-> 1

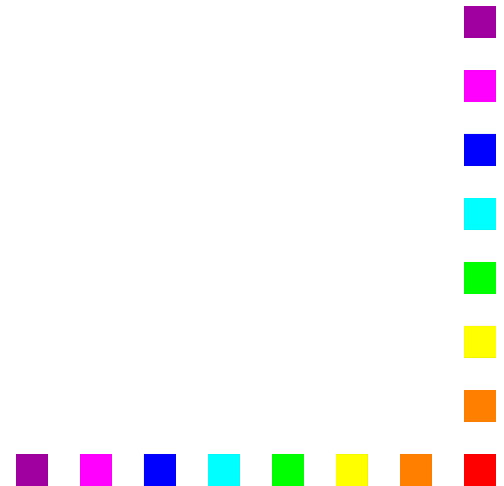
Enter supplier key (2 digits)-> 05

05	YACHTING MONTHLY	TREE HOUSE, LONDON, ENGLAND
----	------------------	-----------------------------

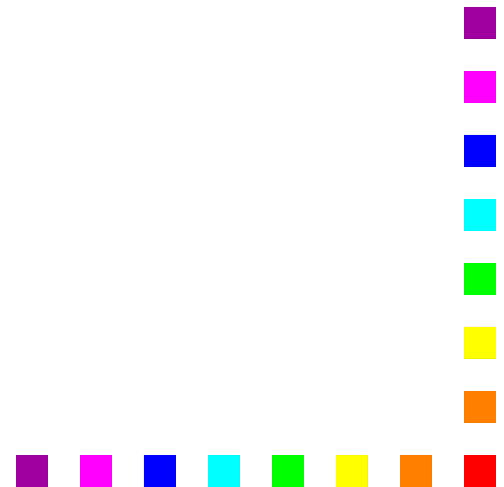
Select and Assign for Relative Files



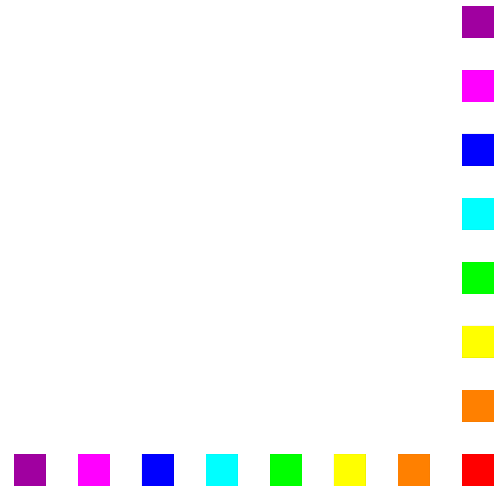
FDs for Relative Files



Relative File Verbs - OPEN

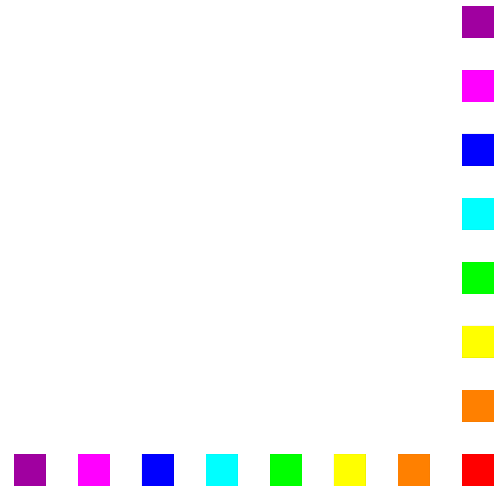


Relative File Verbs - READ

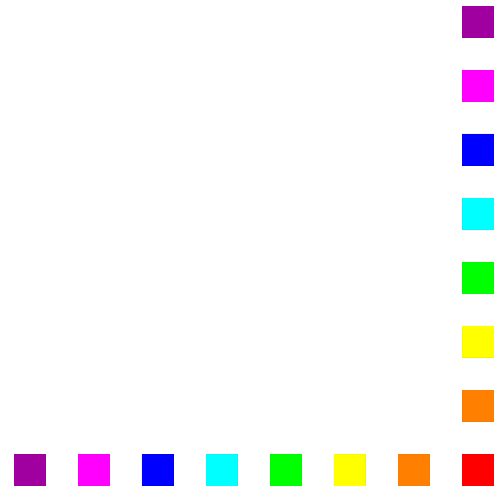


Relative File Verbs - Write and Rewrite

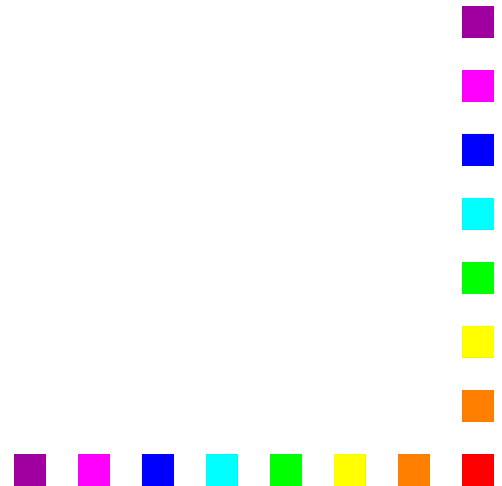
```
REWRITE RecName [FROM SourceItem]  
[INVALID KEY StatementBlock]  
[END-REWRITE]
```



Relative File Verbs - DELETE



Relative File Verbs - START



Error Handling Using Declaratives.

PROCEDURE DIVISION.

DECLARATIVES.

SectionOne SECTION.

USE clause for this section.

ParOne1.

????????????????
????????????????

USE [GLOBAL] AFTER STANDARD

{EXCEPTION
ERROR} PROCEDURE ON

{FileName ...
INPUT
OUTPUT
I-O
EXTEND}

ParOne2.

????????????????
????????????????

SectionTwo SECTION.

USE clause for this section.

ParTwo1.

????????????????
????????????????

ParTwo2.

????????????????
????????????????

END-DECLARATIVES.

Main SECTION.

Begin.

Error Handling Using Declaratives.

```
PROCEDURE DIVISION.  
DECLARATIVES.  
FileError SECTION.  
    USE AFTER ERROR PROCEDURE ON RelativeFile.  
CheckFileStatus.  
    EVALUATE TRUE  
        WHEN RecordDoesNotExist  
            DISPLAY "Record does not exist"  
        WHEN RecordAlreadyExists  
            DISPLAY "Record already exists"  
        WHEN FileNotOpen OPEN I-O RelativeFile  
    END-EVALUATE.  
END-DECLARATIVES.  
Main SECTION.  
Begin.
```